

Documentation-Driven Threat Analysis

Documentation Authoring Guide

Revision 7 - Rebuild R4

Nuova costruzione della guida DDTA per la scrittura della documentazione

Il documento viene ricostruito progressivamente da una baseline vuota. Ogni nuova sezione sarà aggiunta solo dopo revisione del significato metodologico e verifica dell'integrità delle pagine già approvate.

1 Step 0 - Project Problem Framing

RIQUADRO TEMPORANEO - DA CANCELLARE PRIMA DELLA VERSIONE FINALE

Guida di partenza usata come riferimento genealogico della riscrittura:

DDTA_DOCUMENTATION_AUTHORING_GUIDE_R6_CANDIDATE_R2.tex

Questo riferimento serve solo durante la ricostruzione progressiva. Il contenuto della nuova guida non viene importato automaticamente: ogni regola viene riesaminata mentre viene applicata al caso DermaTriage.

1.1 Che cos'è il framing

Il **Project Problem Framing** mette a fuoco il problema — o la classe di problemi — che il progetto deve affrontare e il contributo principale che intende portare entro un boundary governato. Non è un tipo documentale L1: resta una **precondizione dell'autoring DDTA** e non sostituisce MacroRequirement, Decision o FunctionalRequirement. Il boundary può contenere responsabilità che verranno rese esplicite solo nei MacroRequirement successivi. Stare dentro il boundary non significa che ogni responsabilità, attore, integrazione o fase del ciclo di vita debba comparire nella frase finale di framing. Il framing deve rendere riconoscibile il problema; non deve anticipare il catalogo delle responsabilità del progetto.

Domanda fondamentale

Quale problema o classe di problemi deve affrontare il progetto, entro quale boundary significativo, se rimuoviamo temporaneamente le scelte di soluzione correnti?

1.2 Regole di authoring

Quello che va scritto è il problema, non la pipeline o il modello scelto per risolverlo. Si può indicare il risultato atteso a livello di problema, ma senza trasformarlo in una sequenza di requisiti. Le regole operative, per esempio una soglia o un fallback, restano fuori: appartengono a livelli successivi. Ogni significato aggiunto deve essere supportato dalle fonti, e aggiungere dettaglio per riempire i campi non serve.

Regola di neutralizzazione della soluzione

Per riconoscere il problema si mettono *temporaneamente* da parte le scelte tecniche correnti. Questo non autorizza a perdere informazione: le scelte realmente governate verranno recuperate al semantic owner corretto nei livelli successivi.

1.3 Procedura umana di costruzione

1. **Leggere le fonti autorizzate prima di scrivere.** Identificare frasi che descrivono scopo, problema, outcome di dominio e boundary, senza assumere che ogni dettaglio sorgente appartenga al framing.
2. **Separare problema e soluzione.** Mettere temporaneamente da parte i dettagli di realizzazione, per esempio una pipeline o una soglia, come possibili contenuti di livello successivo.
3. **Formulare il problema in linguaggio di dominio.** La frase deve restare vera anche se le scelte tecniche correnti vengono sostituite, salvo che una specifica tecnologia costituisca essa stessa un vincolo governato del problema.
4. **Controllare il boundary senza trasformarlo nel framing.** Non ampliare il progetto con obiettivi, attori o responsabilit  che le fonti non autorizzano. Allo stesso tempo, non copiare automaticamente nel framing finale tutto cio' che appartiene al boundary: una responsabilit  puo' essere interna al progetto e restare correttamente rinviata al livello MacroRequirement.
5. **Fermarsi quando il problema e' riconoscibile.** MacroRequirement, Decision e FunctionalRequirement arrivano dopo.

Gate di accettazione del framing

Prima di accettare il testo verificare:

- se tutte le scelte tecniche correnti cambiassero, il problema descritto continuerebbe a esistere?
- il testo descrive un problema/contributo di progetto oppure sta gia' prescrivendo una soluzione?
- sono stati introdotti significati non supportati dalle fonti autorizzate?
- ci sono dettagli che possono essere rimandati senza perdere l'identita' del problema?

Se una risposta segnala contaminazione o informazione non autorizzata, occorre riformulare prima di procedere ai MacroRequirement.

Anti-pattern principali

- trasformare l'architettura corrente nella definizione del problema;
- elencare dettagli tecnici solo per rendere il framing piu' concreto;
- anticipare fallback o regole condizionali per spiegare casi speciali;
- copiare lo stesso significato che verra' poi riscritto integralmente nel primo MacroRequirement;
- introdurre obiettivi plausibili di dominio che le fonti non governano.

DermaTriage - framing ottenuto durante l'applicazione della guida

Il problema che DermaTriage deve affrontare e' il supporto al triage precoce di casi dermatologici relativi a lesioni potenzialmente oncologiche. Il progetto parte dalle informazioni gia' disponibili sul caso per determinarne l'urgenza e una priorita' operativa di presa in carico, con l'obiettivo di favorire un instradamento specialistico tempestivo.

Nell'esempio non compaiono dettagli di realizzazione come la pipeline a quattro stadi o il fallback senza immagine. Queste informazioni non vengono eliminate dal progetto: vengono rinviate al livello documentale che ne possiede il significato.

1.4 Profilo di esecuzione LLM

Il profilo LLM adatta la guida umana all'esecuzione automatica. Non e' una metodologia alternativa e non estende l'autorità DDTA. Il suo scopo e' contenere alcune tendenze ricorrenti dei modelli linguistici: completare oltre il necessario, anticipare la soluzione, riempire i campi per inerzia e spacciare una parafrasi per decomposizione.

1.4.1 Prompt canonico - Project Problem Framing

ROLE

You are helping with DDTA documentation authoring. The DDTA Documentation Authoring Guide is the methodological authority. Work only from the authorized project sources provided for this step. Do not import project truth from domain knowledge, earlier reconstructions, Base Analysis, threat models, tooling requirements, or implementation assumptions.

TASK

Derive the Project Problem Framing.

Framing is a precondition for authoring, not an L1 document type. Identify the problem or class of problems the project must address, together with the contribution it intends to make, while setting aside current solution choices.

SOURCE-FIRST RULE

1. Pull out statements supported by the sources that describe purpose, problem, domain outcome, actors, and meaningful boundary.
2. Separate problem meaning from current realization details.
3. Keep realization details, but mark them DEFERRED so they can be placed at the right semantic level later.

WHAT NOT TO PUT IN THE FINAL FRAMING

- names of components, models, algorithms, or protocols just because they exist;
- pipeline stages or architecture choices;
- thresholds, mappings, fallback rules, or interface details;
- responsibilities or goals the authorized sources do not support;
- details added only to make the framing look complete.

COUNTERFACTUAL CHECK

Ask: "If all current technical and architectural choices were replaced, would the stated project problem still exist?" If not, rework the framing unless the relevant choice is itself a governed problem boundary.

STOP RULE

Stop once the project problem and contribution are recognizable. Do not pre-write MacroRequirements, Decisions, or FunctionalRequirements.

OUTPUT

Return exactly the analysis schema and final-output schema defined in this section. Do not omit required fields. For REQUIRED-SLOT fields with no supported value, use "-".

Regola per l'LLM

Un'informazione presente nella source non appartiene automaticamente al framing. Il modello deve prima classificarne il semantic level: conoscere un dettaglio non e' una ragione sufficiente per anticiparlo.

Anche un significato correttamente incluso nel project boundary non appartiene automaticamente alla frase finale di framing. Se puo' essere rinviato ai MacroRequirement senza perdere l'identita' del problema, deve restare fuori dal framing finale.

1.5 Schema di output per l'LLM

Per il framing si tengono separati due livelli: la **worksheet di analisi** e l'**output documentale finale**. La worksheet serve a rendere verificabile il ragionamento del modello; non va copiata nella documentazione governata.

1.5.1 Classificazione dei campi

Campo	Classe	Regola
Authorized sources	REQUIRED-CONTENT	Elenco delle sole fonti autorizzate per il passo.
Source-supported problem meaning	REQUIRED-CONTENT	Fatti di problema direttamente sostenuti dalle fonti.
Deferred solution details	REQUIRED-SLOT	Dettagli riconosciuti ma rinviati; "-" se assenti.
Unsupported / ambiguous claims	REQUIRED-SLOT	Significati non autorizzati o ancora ambigui; "-" se assenti.
Meaningful boundary	REQUIRED-SLOT	Boundary governato; "-" se non stabilito separatamente.
Counterfactual result	REQUIRED-CONTENT	PASS o REWORK con motivazione breve.
Final framing	REQUIRED-CONTENT	Un blocco conciso di prosa governata.

1.5.2 Formato esatto richiesto

```
[DDTA FRAMING ANALYSIS]
AUTHORIZED SOURCES:
- ...

SOURCE-SUPPORTED PROBLEM MEANING:
- ...

MEANINGFUL BOUNDARY:
-

DEFERRED SOLUTION DETAILS:
- ...

UNSUPPORTED / AMBIGUOUS CLAIMS:
- ...

COUNTERFACTUAL CHECK:
PASS | REWORK
Rationale: ...

FRAMING VERDICT:
PROCEED | REWORK | STOP
Rationale: ...

[FINAL PROJECT PROBLEM FRAMING]
<one concise governed prose block>
```

1.6 Gate sull'output LLM

Controlli prima di accettare l'output LLM

- il blocco finale contiene solo significato già verificato nella worksheet;
- i dettagli marcati DEFERRED non ricompaiono nel framing finale senza una ragione esplicita;
- nessun campo vuoto viene riempito con una parafrasi di un altro campo;
- PROCEED è ammesso solo se il counterfactual check e la source-authority review sono superati;
- l'output LLM resta una proposta da sottoporre alla stessa review semantica prevista per l'autore umano.

Nota sui controlli quantitativi

Le metriche consolidate di similarità e retrieval del corpus (Jaccard su shingles, cosine TF-IDF, cosine su embeddings e BM25 per retrieval) verranno definite in una sezione metodologica dedicata. Non sostituiscono i gate semantici e non sono necessarie per accettare il primo framing di un corpus privo di altri elementi documentali.

Separazione tra diagnostica e decisione metodologica

Le metriche quantitative potranno segnalare sovrapposizioni o possibili rewording, ma non potranno decidere da sole se un elemento DDTA è corretto. La decisione resta affidata ai gate semantici previsti dalla guida.

2 Uso della metodologia: authoring nativo e reconstruction

DDTA nasce come metodologia per l'**authoring nativo della documentazione di progetto**. Il suo impiego naturale comincia insieme al progetto e ne accompagna la progressiva definizione: problema, responsabilit , decisioni, requisiti.

Ricostruire documentazione preesistente   invece un uso secondario, di validazione: serve a recuperare il significato governabile dalle fonti disponibili e a far emergere ambiguit , lacune e sovrapposizioni che una documentazione DDTA nativa avrebbe dovuto prevenire o quantomeno rendere esplicite.

Tenere separati i due percorsi

Le regole per leggere documentazione legacy non devono appesantire il percorso di authoring nativo.

Percorso	Punto di partenza e uso
NATIVE	Si parte dal problema governato e dalle responsabilit� che il progetto deve assumere. Le scelte di soluzione vengono introdotte ai livelli successivi quando diventano commitment governati.
RECONSTRUCTION	Si parte da fonti legacy autorizzate e si recupera soltanto il significato che esse supportano. Ambiguit�, conflitti e significati mancanti restano visibili invece di essere completati per inferenza.

Stessa semantica, diverso punto di ingresso

I costrutti DDTA non cambiano tra i due percorsi. Cambia il modo in cui si ottiene il significato da documentare: nel native authoring viene governato mentre il progetto prende forma; nella reconstruction viene recuperato da una documentazione che non era stata scritta secondo DDTA.

3 Step 1 - MacroRequirement

Un MacroRequirement copre una sola **responsabilita' macro durevole che il progetto deve possedere per affrontare il problema governato**. Non dice ancora come quella responsabilita' verra' realizzata, e non e' un contenitore per ogni capability importante della soluzione. Una capability non diventa MacroRequirement solo perche' e' stabile, distinguibile o tecnicamente sostituibile. Occorre verificare che il progetto debba possedere quel significato come responsabilita' macro, e non soltanto che la soluzione corrente abbia bisogno di un passaggio equivalente.

Domanda fondamentale

Quale responsabilita' macro deve assumersi il progetto per affrontare il problema governato? Che contributo stabile porta al problema o al boundary? Il progetto deve possedere proprio questa responsabilita', oppure essa esiste come capability distinguibile solo perche' la soluzione corrente ha scelto di decomporre il problema in questo modo? Quale parte del suo significato resta necessaria anche se cambia l'architettura con cui il problema viene affrontato?

Il framing non e' un catalogo di MacroRequirement. Un MR deve restare **riconducibile al problema e al boundary governato**, ma non serve che compaia parola per parola nella frase di framing. Se una nuova responsabilita' cambia davvero il problema o il boundary, allora si riapre il framing.

MR e scelte di soluzione

Una Decision puo' far nascere obblighi e capability reali senza per questo creare nuove responsabilita' macro. Perche' un elemento diventi MR, il suo significato deve essere governato e posseduto dal progetto al livello del problema o del boundary, indipendentemente dalla particolare decomposizione scelta per realizzarlo. La semplice possibilita' di sostituire il componente che oggi realizza una capability non e' sufficiente. Se una diversa architettura plausibile puo' affrontare lo stesso problema senza conservare quella capability come responsabilita' distinguibile, il candidato e' probabilmente una Decision, un FunctionalRequirement o un significato di livello successivo.

Confine con Decision

Una Decision restringe un solo MR fissando una posizione progettuale significativa: per esempio una policy, un responsibility boundary, una strategia o una scelta tecnologica/architetturale con conseguenze downstream. Se il candidato descrive gia' *come* una responsabilita' viene governata o realizzata attraverso una posizione di questo tipo, va valutato come Decision e non come nuovo MR.

Due errori opposti

Da una parte, non ogni sotto-problema introdotto dalla soluzione merita di diventare MR. Dall'altra, responsabilita' macro diverse non vanno compresse in un unico MR solo perche' oggi le realizza lo stesso componente o la stessa pipeline.

3.1 Procedura umana: dalla responsabilita' candidata al MR

Si parte dal framing e dal boundary: quali responsabilita' macro deve possedere il progetto per affrontare quel problema? La risposta non deve anticipare una soluzione. Nel native authoring i candidati emergono dal lavoro di definizione; nella reconstruction arrivano solo dalle fonti legacy autorizzate. In questa fase una lista anche sovrabbondante va bene.

Ogni candidato riceve un **Title** e un **Intent**. Se serve, si aggiungono Context o Scope, ma solo quando aiutano davvero a capire di quale responsabilita' si sta parlando. Prima di valutarne la forma, chiedere se il progetto deve possedere quel significato **come responsabilita' macro**, oppure se il candidato descrive una capability che esiste perche' una particolare Decision o decomposizione architetturale ha introdotto quel passaggio. Nel secondo caso il significato va preservato, ma valutato a un livello inferiore. In **RECONSTRUCTION** questa ownership non deve essere dichiarata letteralmente dalla fonte: deve risultare dal significato che la fonte assegna stabilmente al progetto, senza completarlo per inferenza. I candidati si confrontano poi tra loro: overlap, containment, possibili dipendenze e diversa profondita' di decomposizione contano piu' della descrizione isolata del singolo candidato. Prima di compilare il documento si classifica: KEEP, MERGE, SPLIT, LOWER LEVEL, REWORK oppure DEFER. Solo i KEEP passano alla stesura completa.

Forma di lavoro minima per un candidato

Candidate title:

...

Intent:

...

Context:

[se necessario per disambiguare]

...

Scope note:

[se necessario per separare ownership]

...

Possible dependency:

[se emerge una dipendenza non gerarchica]

...

Questa e' una worksheet di authoring, non un nuovo tipo documentale L1 e non sostituisce il MacroRequirement finale.

3.2 Test di separazione e classificazione

Prima di accettare un candidato come MR, verificarlo con un insieme di domande complementari. Nessuna singola domanda decide da sola il livello semantico.

Test	Domanda operativa
Contributo al problema	Che contributo macro porta al problema o al boundary?
Resilienza alla soluzione	Se venisse adottata un'architettura plausibile diversa per affrontare lo stesso problema, questa responsabilit� continuerebbe a esistere come responsabilit� macro distinguibile? Oppure esiste solo perche' l'architettura corrente ha introdotto quel particolare passaggio?
Singularita'	E' una responsabilit� sola o ne sta nascondendo due che potrebbero cambiare separatamente?
Valore distinto	Ha un significato proprio, oppure riscrive il framing o duplica un altro MR?
Dependency vs containment	Il significato governato di questa responsabilit� richiede il contratto semantico di un'altra responsabilit�, pur mantenendo ownership distinta? Oppure esiste soltanto un rapporto di containment, ordine operativo, dataflow o consumo di output?
Coerenza della futura famiglia	Le Decision che la restringerebbero appartengono allo stesso owner semantico? Il fatto che oggi non siano ancora definite non e' un problema.
Orizzonte di decomposizione	Le parti del candidato possono essere ristrette dallo stesso insieme di commitment governati, oppure una parte possiede gia' una Decision family supportata mentre un'altra deve fermarsi al livello MR? Una differenza persistente segnala un possibile hidden split; non giustifica il far procedere tutto il candidato insieme.
Stabilit� temporale	Regge oltre una singola release, configurazione o iterazione?

Come leggere il controfattuale

Non stiamo cercando una responsabilit  che esisterebbe in ogni soluzione immaginabile. Il controfattuale deve pero' essere abbastanza profondo da non limitarsi alla sostituzione locale del componente corrente. Chiedere se "classificare un'immagine" sopravvive al passaggio da EfficientNet a un altro classificatore non basta: entrambe le alternative possono appartenere alla stessa decomposizione architetturale. Occorre chiedere se, adottando una diversa architettura plausibile per affrontare lo stesso problema di progetto, quel significato rimarrebbe una responsabilit  macro che il progetto deve possedere. "Retraining EfficientNet-B4" resta chiaramente legato alla soluzione; "evoluzione controllata del comportamento del sistema" resta invece riconoscibile come responsabilit  anche se cambiano tecnologia e meccanismo di adattamento. Superare il test di resilienza e' necessario ma non sufficiente per diventare MR: il candidato deve superare anche il test di ownership macro e gli altri test di separazione. Nel percorso **RECONSTRUCTION** il controfattuale serve solo a verificare la dipendenza del significato dalla decomposizione corrente; non autorizza a progettare o assumere architetture alternative non presenti nelle fonti.

Dipendenza non e' contenimento

Un MR puo' dipendere semanticamente da un altro MR. La presenza di una dipendenza non giustifica il merge se le due responsabilit  mantengono ownership e significato distinti. Ordine di esecuzione, passaggio di dati, consumo di un output o collocazione nella stessa pipeline non sono da soli sufficienti per stabilire dependsOn. La relazione va dichiarata quando il significato governato del MR dipendente fa affidamento sul contratto semantico dell'altro MR. Un indizio forte di dipendenza esiste quando il significato o le condizioni di validita' della responsabilit  dipendente non possono essere espressi correttamente senza assumere il significato governato dall'altro MR. Se la fonte mostra soltanto che un output viene consumato successivamente, senza governare la relazione semantica tra le due responsabilit , dependsOn non va inferito automaticamente.

3.3 Campi del MacroRequirement

Una volta superata la separazione iniziale, il candidato viene scritto nella forma L2 completa. Ogni campo deve aggiungere significato governato; la presenza di uno slot nel template non autorizza a inventare contenuto.

Campo	Classe	Uso corretto
Title	REQUIRED-CONTENT	Nome leggibile della responsabilit� macro. Non un requisito operativo, non una soluzione.
Intent	REQUIRED-CONTENT	Dice quale responsabilit� macro assume il progetto e quale contributo porta.
Context	REQUIRED-CONTENT	Il minimo background per interpretare la responsabilit� senza ambiguit�. Non � una parafrasi dell'Intent.
Stakeholders	REQUIRED-SLOT	Ruoli di dominio che beneficiano della responsabilit�, la governano, la operano o ne sono materialmente coinvolti. "�" se non emerge un significato distinto.
Scope	REQUIRED-SLOT	Boundary IN/OUT, quando serve a prevenire overlap o responsibility leakage. "�" se non aggiunge nulla.
Assumptions / Constraints	REQUIRED-SLOT	Solo condizioni macro valide per l'intero branch. "�" se non ce ne sono.
dependsOn	REQUIRED-SLOT	Dipendenza semantica non gerarchica da un altro MR: il significato della responsabilit� fa affidamento sul contratto governato dell'altro MR. Ordine operativo, dataflow o semplice consumo di output non bastano. None se nessuna dipendenza semantica � governata.
Lifecycle / Authority	governance metadata	Stato e authority del documento, quando il corpus li usa. Non sostituiscono il significato del MR.

Quando un campo non va riempito

Se un campo non porta niente di nuovo, non va riempito riscrivendo Intent o Context con altre parole. Dove l'assenza   ammessa, " "   meglio di una frase di comodo.

Scope non   un elenco di cose escluse

Non serve elencare modelli, pipeline o soglie solo per dire che non appartengono all'MR. Le scelte di soluzione restano semplicemente ai livelli che le governano.

3.4 Template L2 e risultato della review

```
# <MR-ID> - <Title>
Lifecycle: <...>
Authority: <...> [se usato dalla governance del corpus]

## Intent
...
## Context
...
## Stakeholders
...
## Scope
...
## Assumptions / Constraints
...
## dependsOn
<MR-ID or None>
```

A compilazione fatta, split e merge vanno ricontrollati. Output diversi non bastano per tenere due MR separati. Condividere oggi un componente, dei dati o una sequenza operativa non basta per fonderli. Va controllato anche l'**orizzonte di decomposizione**. Se una parte del candidato dispone di commitment governati che permettono di procedere a Decision mentre un'altra parte resta semanticamente corretta solo fermandosi a MR, verificare esplicitamente se il candidato nasconde due responsabilit  distinte. Questa differenza   un segnale di review, non una regola automatica di split: puo' anche riflettere una fonte incompleta. Un commitment governato su una parte non deve trascinare automaticamente l'altra a un livello di decomposizione che la fonte non supporta.

DermaTriage - STOP AT MR

La documentazione supporta una responsabilit  macro di indicazione della destinazione specialistica, ma non governa in modo sufficiente il vocabolario delle destinazioni, la regola di selezione, gli eventuali input specifici, il fallback o la responsabilit  di booking/assignment. La presenza, nello stesso Adaptation Layer, di commitment governati relativi alla priorit  operativa non autorizza a usarli come Decision della responsabilit  specialistica. La risposta corretta   mantenere il significato specialistico disponibile e fermare quel branch: non si inventano Decision o FunctionalRequirement per completare la gerarchia.

Gate D1 - MR

PASS quando la responsabilit    stabile, distinta dagli altri MR, collocata al livello corretto e sufficientemente chiara da non ammettere letture materialmente diverse. Dopo PASS chiedere se, nello scope corrente, esistono ulteriori commitment governati che restringono specificamente questa responsabilit  semanticamente omogenea e consentono di derivarne una Decision senza importare significato da un'altra parte del candidato, da un MR vicino o dalla struttura della soluzione.

NO -> STOP AT MR. YES -> procedere alle Decision.

Un commitment che restringe soltanto una parte di un candidato aggregato non autorizza l'intero candidato a procedere. In quel caso occorre riesaminare prima lo split/merge.

3.5 Sufficienza semantica e diagnosi

Il gate non misura la lunghezza del testo. Chiede se, con il significato governato a disposizione, restano in piedi letture materialmente diverse della responsabilit , del suo boundary o del suo owner. In reconstruction conviene classificare le proposition critiche come:

AFFIRMED
DENIED
NOT SPECIFIED
CONFLICTING

NOT SPECIFIED non   FALSE, e CONFLICTING non autorizza a scegliere la lettura piu' comoda.

Diagnosi	Cosa segnala
Source / documentation gap	Il significato manca, � ambiguo o conflittuale gia' nella fonte.
Authoring-guide problem	Il significato si puo' rappresentare, ma domande e gate non guidano l'autore in modo stabile.
Metamodel pressure	Il significato � chiaro e governato; sono i costrutti DDTA a non riuscire a rappresentarlo.
Representation / L2 issue	L1 basta, ma la forma documentale non � chiara o pratica.
Downstream / BA issue	La documentazione regge, ma una rappresentazione analitica a valle perde significato.
No gap	Differenza presentazionale, oppure dettaglio fuori dallo scope governato.

Il progetto non si corregge per far funzionare la metodologia

Un dubbio emerso in review non autorizza a cambiare documentazione, guida o metamodel. Prima va diagnosticato dove sta il problema.

3.6 Profilo di esecuzione LLM

Il profilo LLM segue gli stessi criteri dell'autore umano. Rende solo espliciti i controlli che servono a evitare tre errori ricorrenti: promuovere a MR un dettaglio di soluzione, chiamare decomposizione una parafrasi, riempire tutti i campi anche dove il significato non e' governato.

3.6.1 Prompt canonico - MacroRequirement

ROLE

You are helping with DDTA documentation authoring. The DDTA Documentation Authoring Guide is the methodological authority. Work only from the project meaning authorized for this authoring step.

AUTHORING MODE

The caller must state one mode: NATIVE or RECONSTRUCTION.

NATIVE: start from the governed project problem, boundary and current project meaning. Implementation state does not become project authority just because it exists.

RECONSTRUCTION: recover project meaning only from the authorized legacy sources. Missing meaning is not repaired with domain knowledge, earlier DDTA reconstructions, Base Analysis, threat models, tooling needs or implementation assumptions.

TASK

Identify, separate, and author MacroRequirements. A MacroRequirement owns one durable macro responsibility the project must possess to address the governed problem within its boundary. It is not a container for every stable capability or important project concern.

CANDIDATE DISCOVERY

1. Identify candidate responsibilities without assuming that every stable capability is a MacroRequirement.
2. Give each candidate at least a Title and Intent before judging it.
3. Add Context, Scope, or a possible dependency only when they are needed to understand ownership or separation.
4. Ask whether the project must own that meaning as a macro responsibility, or whether it exists because the current solution decomposes another responsibility into that capability.
5. Preserve solution-induced obligations, but classify them LOWER_LEVEL when a technical or architectural commitment introduced that decomposition.

SEPARATION TESTS

For every candidate ask:

- What distinct macro contribution does it make to the governed problem or boundary?
- MACRO OWNERSHIP: must the project own this meaning as a macro responsibility, or does it exist because the current solution decomposes another responsibility into this capability? In RECONSTRUCTION, infer this only from source-supported project meaning; do not require literal ownership wording.
- SOLUTION RESILIENCE: if a plausibly different architecture were used to address the same project problem, would this meaning still exist as a distinguishable macro responsibility? Do not test only replacement of the current component by an equivalent component. In RECONSTRUCTION, use this only as a classification counterfactual; do not design or assume unsupported alternative architectures.
- Is it one responsibility, or several responsibilities that could change apart?
- Does it add its own meaning, rather than restating the framing or another MR?
- Is its relation to another MR a semantic dependency, containment, operational sequence, dataflow, or simple output consumption?
- Would the Decisions restricting it belong to one coherent semantic owner?
- DECOMPOSITION HORIZON: can the whole candidate be restricted by the same governed commitment family, or can one part proceed to Decisions while another must STOP AT MR? Treat a mismatch as SPLIT-PRESSURE, not an automatic split.
- Does the responsibility remain stable beyond one release or configuration?

No single PASS makes a candidate a MacroRequirement. Solution resilience or a coherent Decision family is not sufficient without macro ownership.

Do NOT require complete independence from other MR. A valid MR may depend on another MR through dependsOn while retaining separate ownership. Do not infer dependsOn from execution order, dataflow, or output consumption alone.

CLASSIFY BEFORE FULL AUTHORING

KEEP | MERGE | SPLIT | LOWER_LEVEL | REWORK | DEFER

Only KEEP candidates may become final MacroRequirements.

Regola per l'LLM

Conoscere un dettaglio della soluzione non gli assegna automaticamente un posto nel livello MR. Il modello deve prima stabilire quale responsabilit  governa quel significato, se il progetto deve possederlo come responsabilit  macro e se regge a un cambio plausibile della decomposizione architetturale. Un singolo test superato non basta per promuovere il candidato.

FULL MR AUTHORING

For each KEEP candidate produce:

- Title: REQUIRED-CONTENT
- Intent: REQUIRED-CONTENT
- Context: REQUIRED-CONTENT
- Stakeholders: REQUIRED-SLOT; use "-" if no distinct governed meaning exists
- Scope: REQUIRED-SLOT; use "-" if it adds no distinct boundary information
- Assumptions / Constraints: REQUIRED-SLOT; use "-" if none are governed
- dependsOn: REQUIRED-SLOT; use "None" when no semantic dependency is governed

FIELD NOVELTY RULE

Do not fill a field by paraphrasing another field. Scope is not a list of technical choices to exclude. Assumptions/Constraints must apply to the whole MR branch.

dependsOn is semantic and non-hierarchical. Do not infer it from narrative order, pipeline order, dataflow, shared data, or simple consumption of another MR's output. Use dependsOn only when the governed meaning of one MR relies on the semantic contract owned by another MR. A strong indicator exists when the dependent responsibility cannot be expressed correctly without assuming that governed meaning.

SEMANTIC SUFFICIENCY

Ask whether materially different interpretations of the responsibility can still survive. If yes, REWORK or STOP rather than inventing lower-level meaning. A stable MR may PASS even when lower-level Decisions or FR are NOT SPECIFIED.

BRANCH STOP RULE

After an MR passes, proceed to Decisions only when additional governed commitments specifically restrict that semantic responsibility.

A commitment that restricts another MR, or only one separable part of an aggregated candidate, does not authorize this branch to proceed.

If different parts of the candidate have different decomposition horizons, re-open SPLIT/MERGE before deciding PROCEED TO DECISIONS. A mismatch is a review signal, not an automatic split.

Otherwise STOP AT MR.

3.7 Worksheet richiesta all'LLM

La worksheet rende verificabile la classificazione dei candidati e resta separata dalla documentazione governata finale.

3.7.1 Formato esatto richiesto

```
[DDTA MR ANALYSIS]
AUTHORING MODE: NATIVE | RECONSTRUCTION
AUTHORIZED PROJECT BASIS:
- ...

CANDIDATE <n>
TITLE: ...
INTENT: ...
SOURCE / GOVERNED SUPPORT: ...
CONTEXT NOTE: ... | -
SCOPE NOTE: ... | -
POSSIBLE DEPENDENCY: ... | None
MACRO-OWNERSHIP: PASS | LOWER_LEVEL | UNCERTAIN
SOLUTION-RESILIENCE: PASS | REWORK
DISTINCT-RESPONSIBILITY: PASS | MERGE | SPLIT | REWORK
DECOMPOSITION-HORIZON: COHERENT | SPLIT-PRESSURE | NOT-YET-TESTABLE
SEMANTIC-LEVEL: MR | LOWER_LEVEL | UNCERTAIN
CLASSIFICATION: KEEP | MERGE | SPLIT | LOWER_LEVEL | REWORK | DEFER
RATIONALE: ...

[FINAL MACROREQUIREMENTS]
MR <id or candidate-id> - <Title>
Intent: ...
Context: ...
Stakeholders: ... | -
Scope: ... | -
Assumptions / Constraints: ... | -
dependsOn: <MR-ID> | None
MR VERDICT: PASS | REWORK
NEXT: PROCEED TO DECISIONS | STOP AT MR
```

Controlli prima di accettare l'output LLM

Prima di accettare l'output:

- ogni MR finale viene da un candidato classificato KEEP;
- ogni KEEP ha superato il macro-ownership test; solution resilience da sola non basta;
- nessun dettaglio marcato LOWER_LEVEL rientra come responsabilit  macro senza una nuova giustificazione;
- il solution-resilience test considera un cambio plausibile di architettura, senza progettare alternative non autorizzate in reconstruction;
- split e merge sono motivati sul significato, non sulla struttura attuale del software;
- una differenza di decomposition horizon viene riesaminata come possibile split, mai trattata come split automatico;
- dependsOn non viene dedotto dal solo ordine operativo, dataflow o consumo di output;
- Context e Scope portano informazione nuova invece di ripetere l'Intent;
- STOP AT MR si applica localmente alla responsabilit : commitment presenti in un'altra parte del candidato non autorizzano il branch a procedere;
- l'output LLM passa comunque dalla stessa review semantica dell'autore umano.

Indice di integrita' delle pagine

Regola di verifica

Ogni pagina possiede un MD5 del proprio contenuto canonico. Il calcolo usa il contenuto LaTeX compreso tra i marker PAGE-CONTENT; il valore MD5 stampato sulla pagina e' escluso dal payload della pagina stessa. Nell'indice, gli hash delle altre pagine sono risolti prima del calcolo, mentre l'hash della pagina indice viene sostituito dal token canonico <SELF> per evitare una dipendenza circolare.

Pag.	Contenuto	MD5 contenuto
1	Frontespizio della nuova guida di authoring	21a890bd38c7d939aec1bf672e055498
2	Project Problem Framing - definizione e regole di authoring	015c757c3dc4118c78c6d2960996936e
3	Project Problem Framing - procedura umana, gate ed esempio DermaTriage	22e57dfc4b4f586235f8aed3c522a7e2
4	Project Problem Framing - LLM Execution Profile e prompt canonico	3bdac4b5fec97501c23d7968bf1619d5
5	Project Problem Framing - regola LLM e classificazione dei campi	b69ae801bbd9fab0dd9722c5416054c4
6	Project Problem Framing - formato esatto dell'output LLM	dd6326ef067e8650d8c2fb33662caf1b
7	Project Problem Framing - gate LLM e nota sui controlli quantitativi	8828e008581efc3d40954e198bdb8c0c
8	Uso della metodologia - authoring nativo e reconstruction	39dd4da3676b9d5a04cebdd9a196c627
9	MacroRequirement - definizione e rapporto con framing e Decision	ddcc227fc8f2ef02b4235ea1a2979c66
10	MacroRequirement - procedura umana per individuare i candidati	fee9637e13dfb1e7c117390773a8e3ae
11	MacroRequirement - test di separazione, split e merge	97e6e0faea39d999309beffb6aca06bd
12	MacroRequirement - campi documentali e regole di compilazione	70297884e3aacfc8f40b330ff7183ca4
13	MacroRequirement - template, split merge e STOP AT MR	9a97d1490def76a4e8d0cd67a8307a34
14	MacroRequirement - sufficienza semantica e diagnosi dei problemi	b008d2293cbe637ae1268cc43c3b2351
15	MacroRequirement - profilo LLM e prompt canonico parte 1	af9687992a8bac51cb56ea985aeb2ebc
16	MacroRequirement - prompt canonico LLM parte 2	fab634987bb3e565df9c1d1f7ad066d0
17	MacroRequirement - prompt LLM campi e schema di analisi	bc1bb076c245425bda75ebd8a2cb5c6
18	MacroRequirement - formato esatto output LLM e gate finale	a70c8156b1d0d0ddd6c5640bdb2a8564
19	Indice di integrita' delle pagine	dc806b53878103b8a7a5edc56cca56b4

A ogni passo, una modifica intenzionale cambia l'MD5 della pagina interessata. Una variazione inattesa dell'MD5 di una pagina gia' approvata impone la revisione della modifica prima di proseguire.